

SWEST28 EmbLT — 2026/08/27

Dabaoボードであそんでみた

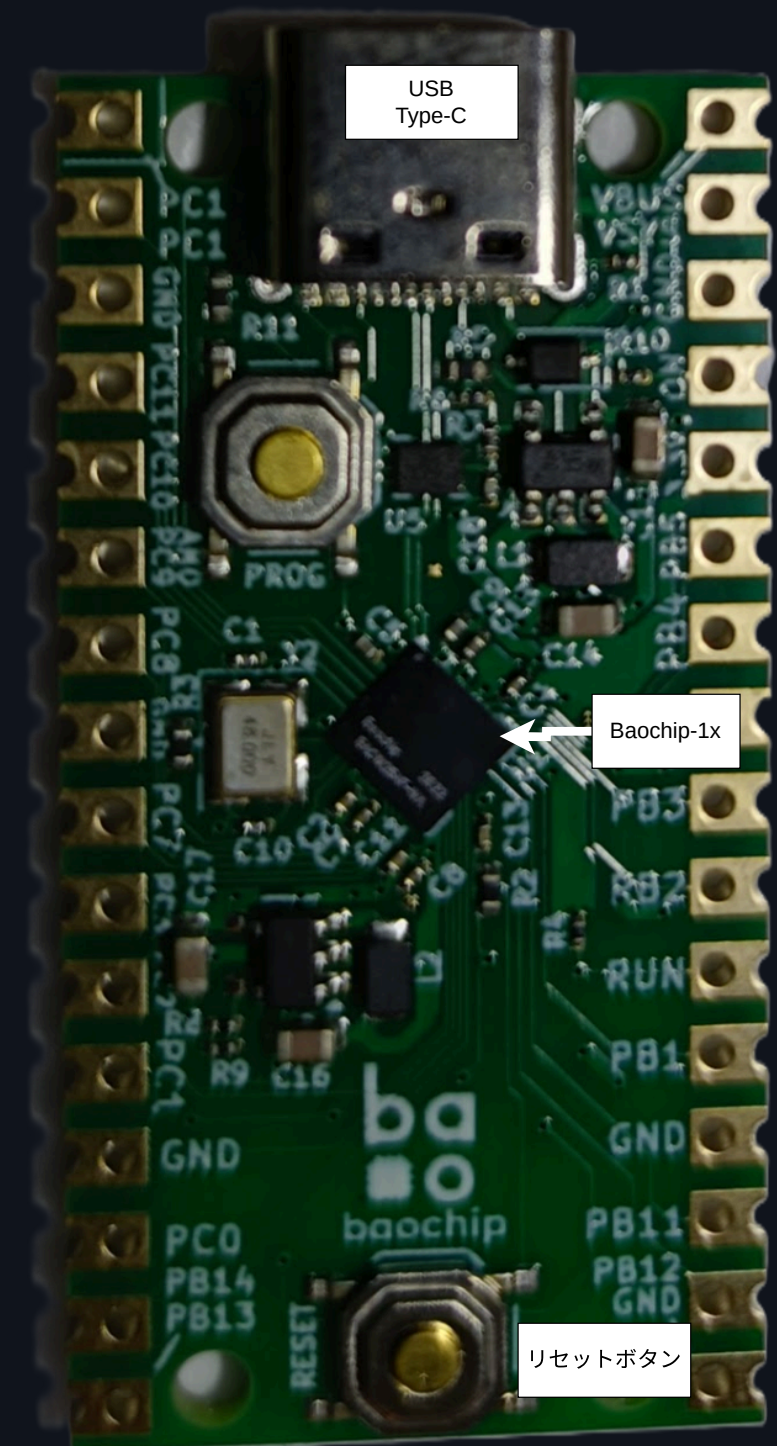
Rust製デバッグプローブ rust-dap を Baochip に移植して、単体で動く GDB デバッガにする

井田 健太 / @ciniml

01 Dabao board / Baochip とは

- ▶ **Baochip-1x** SoC の評価ボード **Dabao**(\$12)
- ▶ 作者は **bunnie**(Andrew "bunnie" Huang)
 - 著書『**ハードウェアハッカー**』(技術評論社)で知られるハードウェアハッカー
- ▶ セキュリティ設計で特徴的な部分
 - **RTL がほぼオープン**(Mostly open RTL)
 - **裏面から赤外線で回路を透視できる**: 実チップが公開 RTL 通りか非破壊で検証できる(バックドア混入の検出)
 - グリッチセンサ、ECC 保護 RAM、セキュアメッシュ、一方向カウンタ

「OSS で検査可能」 = **設計(RTL)が読める** × **実物が設計通りか自分で確かめられる**



02 Baochip-1x の特徴的な構成

CPU コア	VexRiscv RV32IMAC + MMU @ 350MHz
メモリ保護	MPU ではなく MMU (仮想メモリ)
I/O エンジン	BIO : PicoRV コア ×4 @ 700MHz
GPIO 速度	BIO トグルレート 最大 25MHz
メモリ	4MiB RRAM + 2MiB SRAM(オンチップ)

MCU としては珍しい MMU 搭載

全プロセスが独立した仮想メモリ空間。
Rust の所有権・借用のルールを、
プロセス間でも MMU で強制

BIO = I/O コプロセッサ

RP2040 の PIO 相当。ただし専用SMではなく
普通の **RISC-V 命令**で書ける

03 OS は Xous — Pure Rust マイクロカーネル

- ▶ Pure Rust のマイクロカーネル OS
 - 元は秘匿通信端末 Precursor 用に開発
- ▶ すべてが**プロセス分離+メッセージパッシング**
 - ドライバもユーザ空間の「サービス」
 - アプリは IPC 経由でペリフェラルを触る

つまり **GPIO を 1 回叩くたびに IPC が発生する**

04 rust-dap とは

- ▶ Rust 製の CMSIS-DAP デバッグプローブ
ファームウェア
- ▶ Raspberry Pi Pico / XIAO RP2040 /
XIAO M0 などに対応
- ▶ 最近の発展: プローブ単体で GDB RSP を喋る
スタンドアロン GDB サーバ
 - Black Magic Probe 相当
 - PC 側にデバッグソフト不要、GDB 直結



05 Baochip への移植



- ▶ コアはボード非依存クレートに切り出し済み
- ▶ Xous アプリとして実装
(`apps-dabao/dabao-gdb`)

☐ 今回書いた部分

☐ ボード非依存(そのまま再利用)

06 Xous アプリならではのハマりどころ

USB シリアルは Xous のサービス

USB シリアルはすでに Xous の**サービス(別プロセス)**として動いており、アプリからの送受信は**そのサービスとの IPC**になる。読みはブロックするが、gdbstub は非ブロック読みを要求(Idle 中にターゲットをポーリングするため)

→ 受信専用スレッドが別コネクションで IPC 待ちし、キューに積む構成に

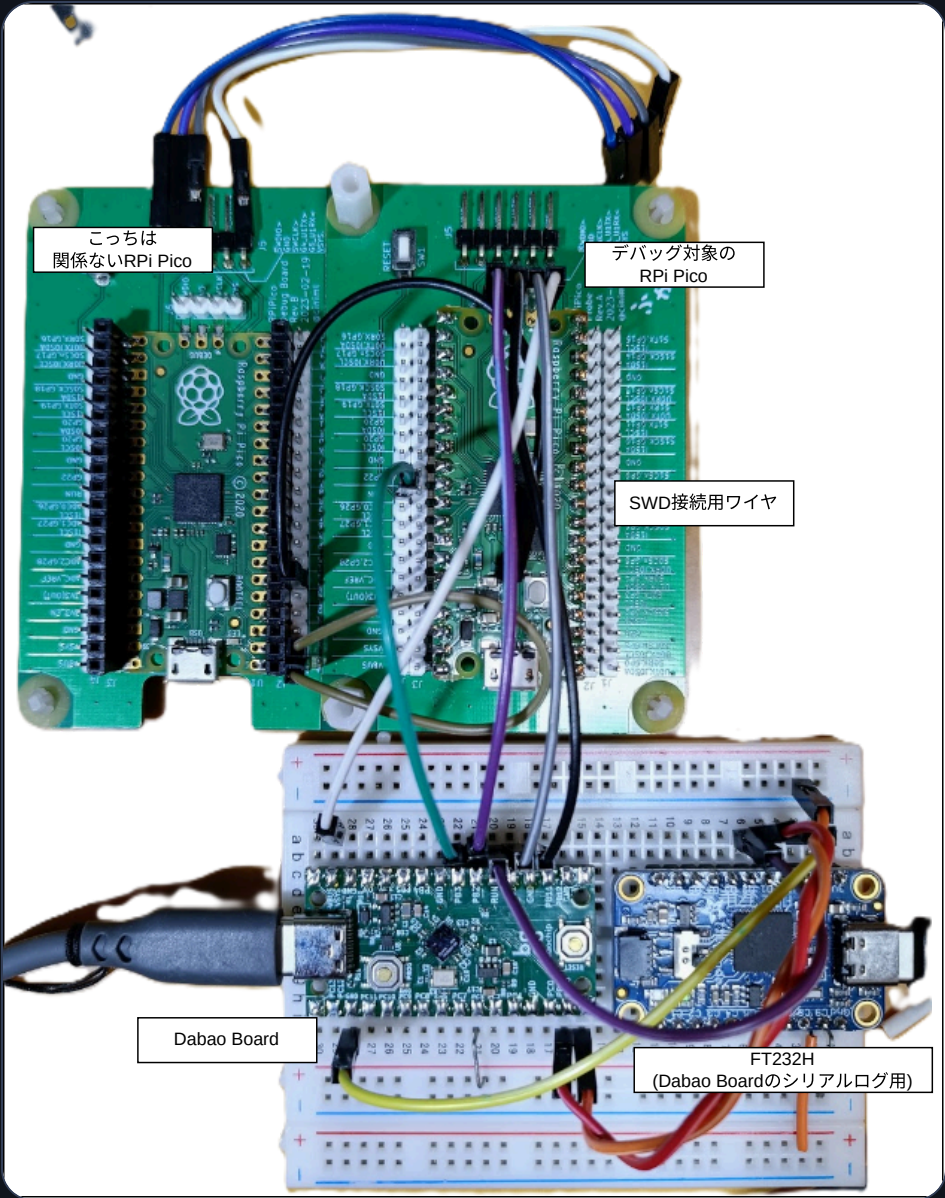
GPIO 操作はマイクロカーネルなので IPC が必要

GPIO は hal-service が所有し、アプリからは **1 操作 = 1 IPC**。

ビットバングのような細かい I/O は BIO に任せる設計思想

→ まずは IPC 越しのビットバング SWD で動かす

07 動かしてみる



Dabao(プローブ)→ SWD → RPi Pico(ターゲット)/ FT232H はログ用 UART

GDB から接続して計測(IPC 越しビットバング SWD)

コマンド	内容	時間
x/1wx ADDR	1 ワード読み	9.1 ms
dump binary memory ...	1 KiB 読み	815 ms (1.23 KiB/s)
dump binary memory ...	16 KiB 読み	13026 ms (1.23 KiB/s)
set {int}ADDR = 0xdeadbeef	1 ワード書き	618 ms
restore ... binary ADDR	4 KiB RAM 書き	3274 ms (1.22 KiB/s)
stepi	ステップ実行	1.1~1.3 s

✔ 接続・halt・レジスタ/メモリ R/W・BP・ステップ、ひと通り動作

08 BIO オフロードで解決した

1.23 KiB/s

IPC 越しビットバング SWD
ワード毎 ~3.2ms $\Rightarrow$ SWCLK 実効 ~30kHz



225 KiB/s

BIO 版 SWD(RAM 読み、SWCLK 8MHz)
約 **180 倍**・入力/ACK/パリティも BIO で完結

項目	IPC 版	BIO 版
attach+detach	0.68 s	0.11 s
1 ワード読み	9.1 ms	0.8 ms
RAM 書き	1.22 KiB/s	207 KiB/s
stepi	1.2 s	55 ms

SWD における BIO の利点

- ▶ **RV32 コア**なので PIO より複雑な処理
(ACK 判定・パリティ計算)を 1 コアに載せられる
- ▶ FIFO CSR を直マップし hot path から IPC を排除
- ▶ 標的からの入力読み・タイミング制御も自前で完結

09 デバッガとして一通り完成

フラッシュ書き込み・RTT まで

- ▶ GDB `load` でフラッシュ書き込み
 - 非同期ローダで **58 KB/s**(compare-sections 一致)
- ▶ **RTT** 対応
 - halt 中: `monitor rtt`、走行中: `0` パケットで
コンソール多重化 **83.8 KiB/s**
- ▶ Pico(PIO SWD)と比較して同等〜上回る

リリース済み(v0.2.1)

- ▶ Dabao 版ファームを GitHub Release に添付
 - `loader / xous / apps.uf2` + ELF
- ▶ `flash.py` 一発で書き込み→起動
- ▶ コア(`gdb-server-core` / `arm-debug`)は Pico と共通 — 両方に同じ修正が効く

`github.com/ciniml/rust-dap` → Releases

10 まとめ

- ▶ Baochip = **MMU 付き RISC-V + BIO** という面白い構成
 - 硬さ(セキュリティ)と速い I/O の両立
- ▶ rust-dap は層分離のおかげで移植コストが小さい
 - 書いたのは Connection / DapTransport / セッションループの3つだけ
- ▶ IPC の遅さは **BIO オフロード** で解決 → **約 180 倍**、フラッシュ load・RTT まで動く
 - Pico(PIO SWD)と同等～上回る速度、v0.2.1 としてリリース済み
- ▶ **BIO はいいぞ!** — RV32 で書ける PIO。SWD に限らず「ちょっと賢い I/O」全般に効く

Dabao は Crowd Supply で予約受付中: crowdsupply.com/baochip/dabao (\$12 / 枚、2026-10-09 出荷予定)

rust-dap のリポジトリ: github.com/ciniml/rust-dap

11 参考文献

- ▶ Baochip / Dabao 紹介(開発者 bunny 氏による一次情報)
 - <https://baochip.com/dabao-intro/>
- ▶ Andrew "bunny" Huang 著、高須正和 訳『ハードウェアハッカー ～新しいモノをつくる破壊と創造の冒険』技術評論社、2018
 - <https://gihyo.jp/book/2018/978-4-297-10106-0>
- ▶ Xous — The Xous Operating System(Xous Book)/ xous-core
 - <https://betrusted.io/xous-book/> / <https://github.com/betrusted-io/xous-core/>
- ▶ Dabao ボード / BIO 関連ツール
 - <https://github.com/baochip/dabao>
 - <https://github.com/baochip/bio-sim> / <https://github.com/baochip/bio-loader> / <https://baochip.com/bio-surfer/>
- ▶ rust-dap
 - <https://github.com/ciniml/rust-dap>